FastAPI Learn Tutorial - User Guide

# Query Parameters and String Validations

**FastAPI** allows you to declare additional information and validation for your parameters.

Let's take this application as example:

Python 3.10+

```python
from fastapi import FastAPI

app = FastAPI()


@app.get("/items/")
async def read_items(q: str | None = None):
    results = {"items": [{"item_id": "Foo"}, {"item_id": "Bar"}]}
    if q:
        results.update({"q": q})
    return results
```

The query parameter `q` is of type `str | None`, that means that it's of type `str` but could also be `None`, and indeed, the default value is `None`, so FastAPI will know it's not required.

> **Note**
>
> FastAPI will know that the value of `q` is not required because of the default value `= None`.
>
> Having `str | None` will allow your editor to give you better support and detect errors.

## Additional validation

We are going to enforce that even though `q` is optional, whenever it is provided, **its length doesn't exceed `50` characters**.

### Import `Query` and `Annotated`

To achieve that, first import:

- `Query` from `fastapi`
- `Annotated` from `typing`

Python 3.10+

```python
from typing import Annotated

from fastapi import FastAPI, Query

app = FastAPI()


@app.get("/items/")
async def read_items(q: Annotated[str | None, Query(max_length=50)] = None):
    results = {"items": [{"item_id": "Foo"}, {"item_id": "Bar"}]}
```

```
    if q:
        results.update({"q": q})
    return results
```

🤓 **Other versions and variants**

**Python 3.10+ - non-Annotated**

**Tip**

Prefer to use the `Annotated` version if possible.

```
from fastapi import FastAPI, Query

app = FastAPI()


@app.get("/items/")
async def read_items(q: str | None = Query(default=None, max_length=50)):
    results = {"items": [{"item_id": "Foo"}, {"item_id": "Bar"}]}
    if q:
        results.update({"q": q})
    return results
```

**Info**

FastAPI added support for `Annotated` (and started recommending it) in version `0.95.0`.

If you have an older version, you would get errors when trying to use `Annotated`.

Make sure you Upgrade the FastAPI version to at least `0.95.1` before using `Annotated`.

## Use `Annotated` in the type for the `q` parameter

Remember I told you before that `Annotated` can be used to add metadata to your parameters in the Python Types Intro?

Now it's the time to use it with FastAPI. 🚀

We had this type annotation:

```
q: str | None = None
```

What we will do is wrap that with `Annotated`, so it becomes:

```
q: Annotated[str | None] = None
```

Both of those versions mean the same thing, `q` is a parameter that can be a `str` or `None`, and by default, it is `None`.

Now let's jump to the fun stuff. 🎉

## Add `Query` to `Annotated` in the `q` parameter

Now that we have this `Annotated` where we can put more information (in this case some additional validation), add `Query` inside of `Annotated`, and set the parameter `max_length` to `50`:

Python 3.10+

```
from typing import Annotated

from fastapi import FastAPI, Query

app = FastAPI()


@app.get("/items/")
async def read_items(q: Annotated[str | None, Query(max_length=50)] = None):
    results = {"items": [{"item_id": "Foo"}, {"item_id": "Bar"}]}
    if q:
        results.update({"q": q})
    return results
```

🤓 **Other versions and variants**

Python 3.10+ - non-Annotated

**Tip**

Prefer to use the `Annotated` version if possible.

```
from fastapi import FastAPI, Query

app = FastAPI()


@app.get("/items/")
async def read_items(q: str | None = Query(default=None, max_length=50)):
    results = {"items": [{"item_id": "Foo"}, {"item_id": "Bar"}]}
    if q:
        results.update({"q": q})
    return results
```

Notice that the default value is still `None`, so the parameter is still optional.

But now, having `Query(max_length=50)` inside of `Annotated`, we are telling FastAPI that we want it to have **additional validation** for this value, we want it to have maximum 50 characters. 😎

**Tip**

Here we are using `Query()` because this is a **query parameter**. Later we will see others like `Path()`, `Body()`, `Header()`, and `Cookie()`, that also accept the same arguments as `Query()`.

FastAPI will now:

- **Validate** the data making sure that the max length is 50 characters
- Show a **clear error** for the client when the data is not valid
- **Document** the parameter in the OpenAPI schema *path operation* (so it will show up in the **automatic docs UI**)

## Alternative (old): `Query` as the default value

Previous versions of FastAPI (before *0.95.0*) required you to use `Query` as the default value of your parameter, instead of putting it in `Annotated`, there's a high chance that you will see code using it around, so I'll explain it to you.

> **Tip**
>
> For new code and whenever possible, use `Annotated` as explained above. There are multiple advantages (explained below) and no disadvantages. 🍰

This is how you would use `Query()` as the default value of your function parameter, setting the parameter `max_length` to `50`:

**Python 3.10+ - non-Annotated**

```python
from fastapi import FastAPI, Query

app = FastAPI()


@app.get("/items/")
async def read_items(q: str | None = Query(default=None, max_length=50)):
    results = {"items": [{"item_id": "Foo"}, {"item_id": "Bar"}]}
    if q:
        results.update({"q": q})
    return results
```

**🤓 Other versions and variants**

**Python 3.10+**

```python
from typing import Annotated

from fastapi import FastAPI, Query

app = FastAPI()


@app.get("/items/")
async def read_items(q: Annotated[str | None, Query(max_length=50)] = None):
    results = {"items": [{"item_id": "Foo"}, {"item_id": "Bar"}]}
    if q:
        results.update({"q": q})
    return results
```

As in this case (without using `Annotated`) we have to replace the default value `None` in the function with `Query()`, we now need to set the default value with the parameter `Query(default=None)`, it serves the same purpose of defining that default value (at least for FastAPI).

So:

```python
q: str | None = Query(default=None)
```

...makes the parameter optional, with a default value of `None`, the same as:

```python
q: str | None = None
```

But the `Query` version declares it explicitly as being a query parameter.

Then, we can pass more parameters to `Query`. In this case, the `max_length` parameter that applies to strings:

```
q: str | None = Query(default=None, max_length=50)
```

This will validate the data, show a clear error when the data is not valid, and document the parameter in the OpenAPI schema *path operation*.

### `Query` as the default value or in `Annotated`

Keep in mind that when using `Query` inside of `Annotated` you cannot use the `default` parameter for `Query`.

Instead, use the actual default value of the function parameter. Otherwise, it would be inconsistent.

For example, this is not allowed:

```
q: Annotated[str, Query(default="rick")] = "morty"
```

...because it's not clear if the default value should be `"rick"` or `"morty"`.

So, you would use (preferably):

```
q: Annotated[str, Query()] = "rick"
```

...or in older code bases you will find:

```
q: str = Query(default="rick")
```

### Advantages of `Annotated`

**Using `Annotated` is recommended** instead of the default value in function parameters, it is **better** for multiple reasons. 🤓

The **default** value of the **function parameter** is the **actual default** value, that's more intuitive with Python in general. 😌

You could **call** that same function in **other places** without FastAPI, and it would **work as expected**. If there's a **required** parameter (without a default value), your **editor** will let you know with an error, **Python** will also complain if you run it without passing the required parameter.

When you don't use `Annotated` and instead use the **(old) default value style**, if you call that function without FastAPI in **other places**, you have to **remember** to pass the arguments to the function for it to work correctly, otherwise the values will be different from what you expect (e.g. `QueryInfo` or something similar instead of `str`). And your editor won't complain, and Python won't complain running that function, only when the operations inside error out.

Because `Annotated` can have more than one metadata annotation, you could now even use the same function with other tools, like Typer. 🚀

## Add more validations

You can also add a parameter `min_length`:

Python 3.10+

```
from typing import Annotated

from fastapi import FastAPI, Query

app = FastAPI()
```

```python
@app.get("/items/")
async def read_items(
    q: Annotated[str | None, Query(min_length=3, max_length=50)] = None,
):
    results = {"items": [{"item_id": "Foo"}, {"item_id": "Bar"}]}
    if q:
        results.update({"q": q})
    return results
```

**Other versions and variants**

Python 3.10+ - non-Annotated

**Tip**

Prefer to use the `Annotated` version if possible.

```python
from fastapi import FastAPI, Query

app = FastAPI()


@app.get("/items/")
async def read_items(q: str | None = Query(default=None, min_length=3, max_length=50)):
    results = {"items": [{"item_id": "Foo"}, {"item_id": "Bar"}]}
    if q:
        results.update({"q": q})
    return results
```

## Add regular expressions

You can define a *regular expression* `pattern` that the parameter should match:

Python 3.10+

```python
from typing import Annotated

from fastapi import FastAPI, Query

app = FastAPI()


@app.get("/items/")
async def read_items(
    q: Annotated[
        str | None, Query(min_length=3, max_length=50, pattern="^fixedquery$")
    ] = None,
):
    results = {"items": [{"item_id": "Foo"}, {"item_id": "Bar"}]}
    if q:
        results.update({"q": q})
    return results
```

🤓 **Other versions and variants**

Python 3.10+ - non-Annotated

**Tip**

Prefer to use the `Annotated` version if possible.

```python
from fastapi import FastAPI, Query

app = FastAPI()


@app.get("/items/")
async def read_items(
    q: str | None = Query(
        default=None, min_length=3, max_length=50, pattern="^fixedquery$"
    ),
):
    results = {"items": [{"item_id": "Foo"}, {"item_id": "Bar"}]}
    if q:
        results.update({"q": q})
    return results
```

This specific regular expression pattern checks that the received parameter value:

- `^`: starts with the following characters, doesn't have characters before.
- `fixedquery`: has the exact value `fixedquery`.
- `$`: ends there, doesn't have any more characters after `fixedquery`.

If you feel lost with all these **"regular expression"** ideas, don't worry. They are a hard topic for many people. You can still do a lot of stuff without needing regular expressions yet.

Now you know that whenever you need them you can use them in **FastAPI**.

## Default values

You can, of course, use default values other than `None`.

Let's say that you want to declare the `q` query parameter to have a `min_length` of `3`, and to have a default value of `"fixedquery"`:

Python 3.10+

```python
from typing import Annotated

from fastapi import FastAPI, Query

app = FastAPI()


@app.get("/items/")
async def read_items(q: Annotated[str, Query(min_length=3)] = "fixedquery"):
    results = {"items": [{"item_id": "Foo"}, {"item_id": "Bar"}]}
    if q:
        results.update({"q": q})
    return results
```

🤓 **Other versions and variants**

Python 3.10+ - non-Annotated

**Tip**

Prefer to use the `Annotated` version if possible.

```
from fastapi import FastAPI, Query

app = FastAPI()


@app.get("/items/")
async def read_items(q: str = Query(default="fixedquery", min_length=3)):
    results = {"items": [{"item_id": "Foo"}, {"item_id": "Bar"}]}
    if q:
        results.update({"q": q})
    return results
```

**Note**

Having a default value of any type, including `None`, makes the parameter optional (not required).

## Required parameters

When we don't need to declare more validations or metadata, we can make the `q` query parameter required just by not declaring a default value, like:

```
q: str
```

instead of:

```
q: str | None = None
```

But we are now declaring it with `Query`, for example like:

```
q: Annotated[str | None, Query(min_length=3)] = None
```

So, when you need to declare a value as required while using `Query`, you can simply not declare a default value:

Python 3.10+

```
from typing import Annotated

from fastapi import FastAPI, Query

app = FastAPI()


@app.get("/items/")
async def read_items(q: Annotated[str, Query(min_length=3)]):
    results = {"items": [{"item_id": "Foo"}, {"item_id": "Bar"}]}
    if q:
        results.update({"q": q})
    return results
```

🤓 **Other versions and variants**

Python 3.10+ - non-Annotated

**Tip**

Prefer to use the `Annotated` version if possible.

```
from fastapi import FastAPI, Query

app = FastAPI()


@app.get("/items/")
async def read_items(q: str = Query(min_length=3)):
    results = {"items": [{"item_id": "Foo"}, {"item_id": "Bar"}]}
    if q:
        results.update({"q": q})
    return results
```

## Required, can be `None`

You can declare that a parameter can accept `None`, but that it's still required. This would force clients to send a value, even if the value is `None`.

To do that, you can declare that `None` is a valid type but simply do not declare a default value:

Python 3.10+

```
from typing import Annotated

from fastapi import FastAPI, Query

app = FastAPI()


@app.get("/items/")
async def read_items(q: Annotated[str | None, Query(min_length=3)]):
    results = {"items": [{"item_id": "Foo"}, {"item_id": "Bar"}]}
    if q:
        results.update({"q": q})
    return results
```

🤓 **Other versions and variants**

Python 3.10+ - non-Annotated

**Tip**

Prefer to use the `Annotated` version if possible.

```
from fastapi import FastAPI, Query

app = FastAPI()


@app.get("/items/")
async def read_items(q: str | None = Query(min_length=3)):
    results = {"items": [{"item_id": "Foo"}, {"item_id": "Bar"}]}
    if q:
        results.update({"q": q})
    return results
```

## Query parameter list / multiple values

When you define a query parameter explicitly with `Query` you can also declare it to receive a list of values, or said in another way, to receive multiple values.

For example, to declare a query parameter `q` that can appear multiple times in the URL, you can write:

Python 3.10+

```
from typing import Annotated

from fastapi import FastAPI, Query

app = FastAPI()


@app.get("/items/")
async def read_items(q: Annotated[list[str] | None, Query()] = None):
    query_items = {"q": q}
    return query_items
```

🤓 **Other versions and variants**

Python 3.10+ - non-Annotated

**Tip**

Prefer to use the `Annotated` version if possible.

```
from fastapi import FastAPI, Query

app = FastAPI()


@app.get("/items/")
async def read_items(q: list[str] | None = Query(default=None)):
    query_items = {"q": q}
    return query_items
```

Then, with a URL like:

```
http://localhost:8000/items/?q=foo&q=bar
```

you would receive the multiple `q` *query parameters'* values (`foo` and `bar`) in a Python `list` inside your *path operation function*, in the *function parameter* `q`.

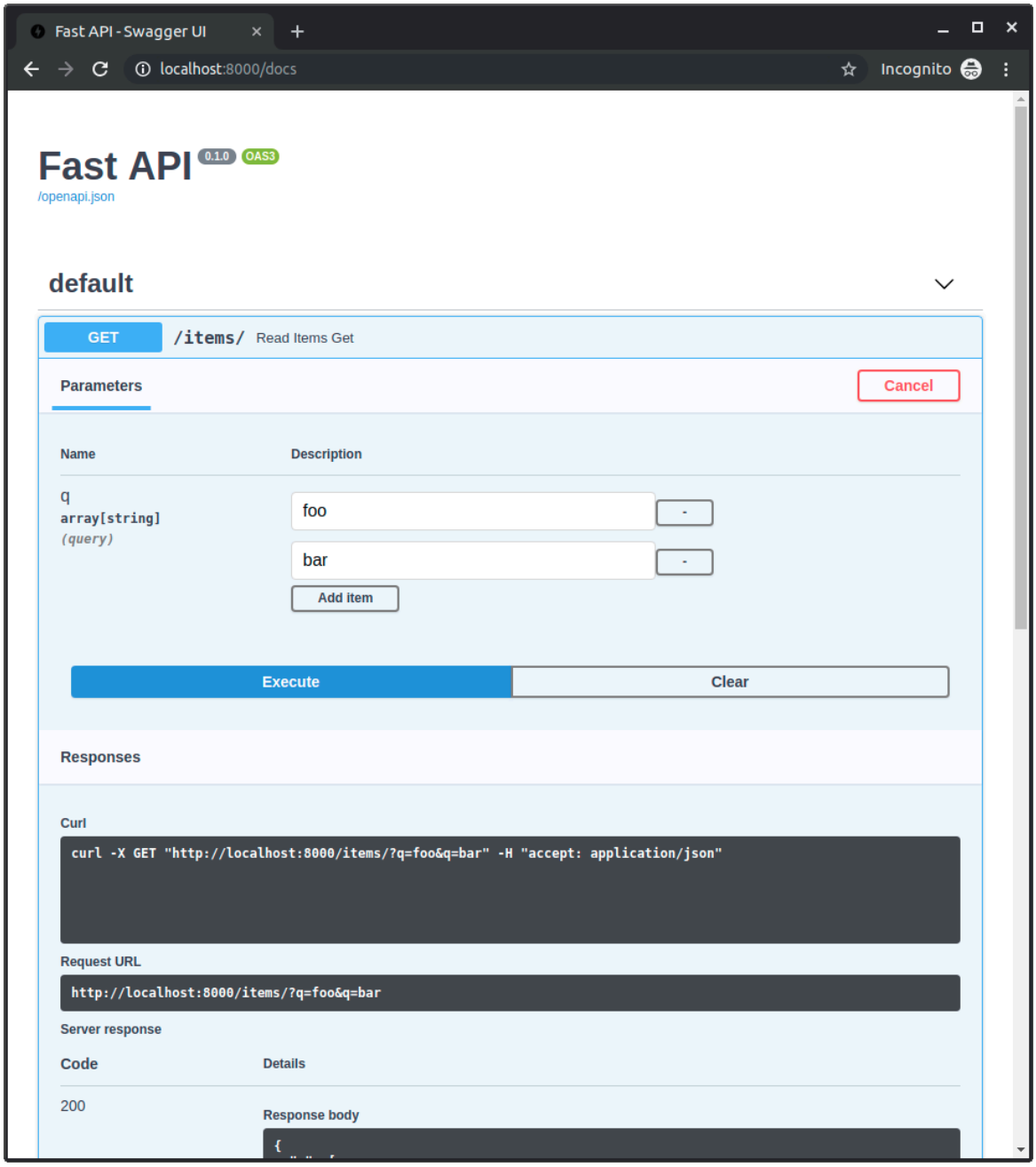
So, the response to that URL would be:

```
{
  "q": [
    "foo",
    "bar"
  ]
}
```

**Tip**

To declare a query parameter with a type of `list`, like in the example above, you need to explicitly use `Query`, otherwise it would be interpreted as a request body.

The interactive API docs will update accordingly, to allow multiple values:




## Query parameter list / multiple values with defaults

You can also define a default `list` of values if none are provided:

Python 3.10+

```python
from typing import Annotated

from fastapi import FastAPI, Query

app = FastAPI()


@app.get("/items/")
async def read_items(q: Annotated[list[str], Query()] = ["foo", "bar"]):
    query_items = {"q": q}
    return query_items
```

🤓 **Other versions and variants**

Python 3.10+ - non-Annotated

**Tip**

Prefer to use the `Annotated` version if possible.

```python
from fastapi import FastAPI, Query

app = FastAPI()


@app.get("/items/")
async def read_items(q: list[str] = Query(default=["foo", "bar"])):
    query_items = {"q": q}
    return query_items
```

If you go to:

```
http://localhost:8000/items/
```

the default of `q` will be: `["foo", "bar"]` and your response will be:

```
{
  "q": [
    "foo",
    "bar"
  ]
}
```

### Using just `list`

You can also use `list` directly instead of `list[str]`:

Python 3.10+

```python
from typing import Annotated

from fastapi import FastAPI, Query

app = FastAPI()


@app.get("/items/")
async def read_items(q: Annotated[list, Query()] = []):
    query_items = {"q": q}
    return query_items
```

🤓 **Other versions and variants**

Python 3.10+ - non-Annotated

**Tip**

Prefer to use the `Annotated` version if possible.

```
from fastapi import FastAPI, Query

app = FastAPI()


@app.get("/items/")
async def read_items(q: list = Query(default=[])):
    query_items = {"q": q}
    return query_items
```

**Note**

Keep in mind that in this case, FastAPI won't check the contents of the list.

For example, `list[int]` would check (and document) that the contents of the list are integers. But `list` alone wouldn't.

## Declare more metadata

You can add more information about the parameter.

That information will be included in the generated OpenAPI and used by the documentation user interfaces and external tools.

**Note**

Keep in mind that different tools might have different levels of OpenAPI support.

Some of them might not show all the extra information declared yet, although in most of the cases, the missing feature is already planned for development.

You can add a `title`:

Python 3.10+

```
from typing import Annotated

from fastapi import FastAPI, Query

app = FastAPI()


@app.get("/items/")
async def read_items(
    q: Annotated[str | None, Query(title="Query string", min_length=3)] = None,
):
    results = {"items": [{"item_id": "Foo"}, {"item_id": "Bar"}]}
    if q:
        results.update({"q": q})
```

```
    return results
```

🤓 **Other versions and variants**

Python 3.10+ - non-Annotated

**Tip**

Prefer to use the `Annotated` version if possible.

```
from fastapi import FastAPI, Query

app = FastAPI()


@app.get("/items/")
async def read_items(
    q: str | None = Query(default=None, title="Query string", min_length=3),
):
    results = {"items": [{"item_id": "Foo"}, {"item_id": "Bar"}]}
    if q:
        results.update({"q": q})
    return results
```

And a `description`:

Python 3.10+

```
from typing import Annotated

from fastapi import FastAPI, Query

app = FastAPI()


@app.get("/items/")
async def read_items(
    q: Annotated[
        str | None,
        Query(
            title="Query string",
            description="Query string for the items to search in the database that have a good
match",
            min_length=3,
        ),
    ] = None,
):
    results = {"items": [{"item_id": "Foo"}, {"item_id": "Bar"}]}
    if q:
        results.update({"q": q})
    return results
```

🤓 **Other versions and variants**

Python 3.10+ - non-Annotated

**Tip**

Prefer to use the `Annotated` version if possible.

```python
from fastapi import FastAPI, Query

app = FastAPI()


@app.get("/items/")
async def read_items(
    q: str | None = Query(
        default=None,
        title="Query string",
        description="Query string for the items to search in the database that have a good match",
        min_length=3,
    ),
):
    results = {"items": [{"item_id": "Foo"}, {"item_id": "Bar"}]}
    if q:
        results.update({"q": q})
    return results
```

## Alias parameters

Imagine that you want the parameter to be `item-query`.

Like in:

```
http://127.0.0.1:8000/items/?item-query=foobaritems
```

But `item-query` is not a valid Python variable name.

The closest would be `item_query`.

But you still need it to be exactly `item-query`...

Then you can declare an `alias`, and that alias is what will be used to find the parameter value:

Python 3.10+

```python
from typing import Annotated

from fastapi import FastAPI, Query

app = FastAPI()


@app.get("/items/")
async def read_items(q: Annotated[str | None, Query(alias="item-query")] = None):
    results = {"items": [{"item_id": "Foo"}, {"item_id": "Bar"}]}
    if q:
        results.update({"q": q})
    return results
```

🤓 **Other versions and variants**

Python 3.10+ - non-Annotated

**Tip**

Prefer to use the `Annotated` version if possible.

```python
from fastapi import FastAPI, Query

app = FastAPI()


@app.get("/items/")
async def read_items(q: str | None = Query(default=None, alias="item-query")):
    results = {"items": [{"item_id": "Foo"}, {"item_id": "Bar"}]}
    if q:
        results.update({"q": q})
    return results
```

## Deprecating parameters

Now let's say you don't like this parameter anymore.

You have to leave it there a while because there are clients using it, but you want the docs to clearly show it as *deprecated*.

Then pass the parameter `deprecated=True` to `Query`:

Python 3.10+

```python
from typing import Annotated

from fastapi import FastAPI, Query

app = FastAPI()


@app.get("/items/")
async def read_items(
    q: Annotated[
        str | None,
        Query(
            alias="item-query",
            title="Query string",
            description="Query string for the items to search in the database that have a good
match",
            min_length=3,
            max_length=50,
            pattern="^fixedquery$",
            deprecated=True,
        ),
    ] = None,
):
    results = {"items": [{"item_id": "Foo"}, {"item_id": "Bar"}]}
    if q:
        results.update({"q": q})
    return results
```

🤓 **Other versions and variants**

Python 3.10+ - non-Annotated

**Tip**

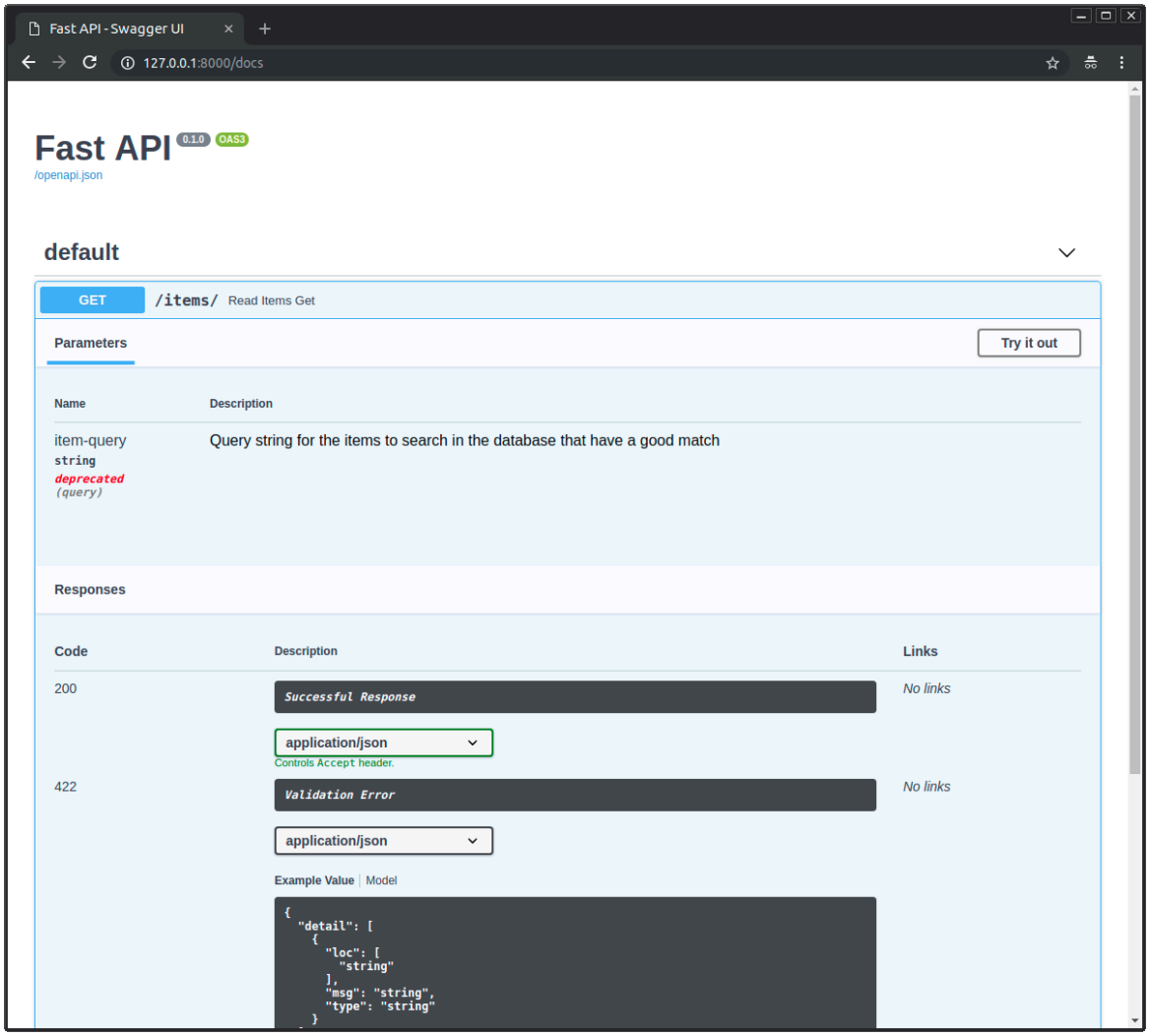
Prefer to use the `Annotated` version if possible.

```
from fastapi import FastAPI, Query

app = FastAPI()


@app.get("/items/")
async def read_items(
    q: str | None = Query(
        default=None,
        alias="item-query",
        title="Query string",
        description="Query string for the items to search in the database that have a good match",
        min_length=3,
        max_length=50,
        pattern="^fixedquery$",
        deprecated=True,
    ),
):
    results = {"items": [{"item_id": "Foo"}, {"item_id": "Bar"}]}
    if q:
        results.update({"q": q})
    return results
```

The docs will show it like this:




## Exclude parameters from OpenAPI

To exclude a query parameter from the generated OpenAPI schema (and thus, from the automatic documentation systems), set the parameter `include_in_schema` of `Query` to `False`:

Python 3.10+

```python
from typing import Annotated

from fastapi import FastAPI, Query

app = FastAPI()


@app.get("/items/")
async def read_items(
    hidden_query: Annotated[str | None, Query(include_in_schema=False)] = None,
):
    if hidden_query:
        return {"hidden_query": hidden_query}
    else:
        return {"hidden_query": "Not found"}
```

🤓 **Other versions and variants**

Python 3.10+ - non-Annotated

**Tip**

Prefer to use the `Annotated` version if possible.

```
from fastapi import FastAPI, Query

app = FastAPI()


@app.get("/items/")
async def read_items(
    hidden_query: str | None = Query(default=None, include_in_schema=False),
):
    if hidden_query:
        return {"hidden_query": hidden_query}
    else:
        return {"hidden_query": "Not found"}
```

## Custom Validation

There could be cases where you need to do some **custom validation** that can't be done with the parameters shown above.

In those cases, you can use a **custom validator function** that is applied after the normal validation (e.g. after validating that the value is a `str`).

You can achieve that using Pydantic's `AfterValidator` inside of `Annotated`.

**Tip**

Pydantic also has `BeforeValidator` and others. 🤓

For example, this custom validator checks that the item ID starts with `isbn-` for an ISBN book number or with `imdb-` for an IMDB movie URL ID:

Python 3.10+

```
import random
from typing import Annotated

from fastapi import FastAPI
from pydantic import AfterValidator

app = FastAPI()

data = {
    "isbn-9781529046137": "The Hitchhiker's Guide to the Galaxy",
    "imdb-tt0371724": "The Hitchhiker's Guide to the Galaxy",
    "isbn-9781439512982": "Isaac Asimov: The Complete Stories, Vol. 2",
}


def check_valid_id(id: str):
    if not id.startswith(("isbn-", "imdb-")):
```

```
        raise ValueError('Invalid ID format, it must start with "isbn-" or "imdb-"')
    return id


@app.get("/items/")
async def read_items(
    id: Annotated[str | None, AfterValidator(check_valid_id)] = None,
):
    if id:
        item = data.get(id)
    else:
        id, item = random.choice(list(data.items()))
    return {"id": id, "name": item}
```

**Info**

This is available with Pydantic version `2` or above. 😎

**Tip**

If you need to do any type of validation that requires communicating with any **external component**, like a database or another API, you should instead use **FastAPI Dependencies**, you will learn about them later.

These custom validators are for things that can be checked with **only** the **same data** provided in the request.

## Understand that Code

The important point is just using **`AfterValidator` with a function inside `Annotated`**. Feel free to skip this part. 🤸

---

But if you're curious about this specific code example and you're still entertained, here are some extra details.

**String with `value.startswith()`**

Did you notice? a string using `value.startswith()` can take a tuple, and it will check each value in the tuple:

Python 3.10+

```
# Code above omitted 👆

def check_valid_id(id: str):
    if not id.startswith(("isbn-", "imdb-")):
        raise ValueError('Invalid ID format, it must start with "isbn-" or "imdb-"')
    return id

# Code below omitted 👇
```

👀 **Full file preview**

Python 3.10+

```python
import random
from typing import Annotated

from fastapi import FastAPI
from pydantic import AfterValidator

app = FastAPI()

data = {
    "isbn-9781529046137": "The Hitchhiker's Guide to the Galaxy",
    "imdb-tt0371724": "The Hitchhiker's Guide to the Galaxy",
    "isbn-9781439512982": "Isaac Asimov: The Complete Stories, Vol. 2",
}


def check_valid_id(id: str):
    if not id.startswith(("isbn-", "imdb-")):
        raise ValueError('Invalid ID format, it must start with "isbn-" or "imdb-"')
    return id


@app.get("/items/")
async def read_items(
    id: Annotated[str | None, AfterValidator(check_valid_id)] = None,
):
    if id:
        item = data.get(id)
    else:
        id, item = random.choice(list(data.items()))
    return {"id": id, "name": item}
```

**A Random Item**

With `data.items()` we get an *iterable object* with tuples containing the key and value for each dictionary item.

We convert this iterable object into a proper `list` with `list(data.items())`.

Then with `random.choice()` we can get a **random value** from the list, so, we get a tuple with `(id, name)`. It will be something like `("imdb-tt0371724", "The Hitchhiker's Guide to the Galaxy")`.

Then we **assign those two values** of the tuple to the variables `id` and `name`.

So, if the user didn't provide an item ID, they will still receive a random suggestion.

...we do all this in a **single simple line**. 🤯 Don't you love Python? 🐍

Python 3.10+

```python
# Code above omitted 👆

@app.get("/items/")
async def read_items(
    id: Annotated[str | None, AfterValidator(check_valid_id)] = None,
):
    if id:
        item = data.get(id)
    else:
        id, item = random.choice(list(data.items()))
    return {"id": id, "name": item}
```

👀 **Full file preview**

Python 3.10+

```python
import random
from typing import Annotated

from fastapi import FastAPI
from pydantic import AfterValidator

app = FastAPI()

data = {
    "isbn-9781529046137": "The Hitchhiker's Guide to the Galaxy",
    "imdb-tt0371724": "The Hitchhiker's Guide to the Galaxy",
    "isbn-9781439512982": "Isaac Asimov: The Complete Stories, Vol. 2",
}


def check_valid_id(id: str):
    if not id.startswith(("isbn-", "imdb-")):
        raise ValueError('Invalid ID format, it must start with "isbn-" or "imdb-"')
    return id


@app.get("/items/")
async def read_items(
    id: Annotated[str | None, AfterValidator(check_valid_id)] = None,
):
    if id:
        item = data.get(id)
    else:
        id, item = random.choice(list(data.items()))
    return {"id": id, "name": item}
```

## Recap

You can declare additional validations and metadata for your parameters.

Generic validations and metadata:

- `alias`
- `title`
- `description`
- `deprecated`

Validations specific for strings:

- `min_length`
- `max_length`
- `pattern`

Custom validations using `AfterValidator`.

In these examples you saw how to declare validations for `str` values.

See the next chapters to learn how to declare validations for other types, like numbers.